Lahore University of Management Sciences

CS4308 Graph Machine Learning

Semester and year to be announced

Proposed research and practice course plan. Administrative details and assessment weights require confirmation.

Instructor	To be announced
Room No.	To be announced
Office Hours	To be announced
Email	To be announced
Telephone	To be announced
Secretary/TA	To be announced
TA Office Hours	To be announced
Course URL (if any)	https://amessbee.github.io/s27/
Support Services	LUMS offers a range of academic and other services to support students. These are mentioned below, and you are encouraged to use these in addition to in-class assistance from course staff. For a complete list of campus support services available for you click here (https://advising.lums.edu.pk/#supportservices)

Course Basics				
Credit Hours	3 (proposed; subject to approval)			
Lecture(s)	Nbr of Lec(s) Per Week	2 (proposed)	Duration	75 minutes (proposed)
Recitation/Lab (per week)	Nbr of Lec(s) Per Week	Integrated practical studios	Duration	Within scheduled meetings
Tutorial (per week)	Nbr of Lec(s) Per Week	Optional TA clinic	Duration	To be announced

Course Distribution	
Core	No - proposed elective
Elective	Yes - proposed CS elective
Open for Student Category	Senior undergraduates; graduate students with permission (proposed)
Close for Student Category	Students without prerequisite preparation, unless instructor permission is granted

COURSE DESCRIPTION

A research and practice course on learning from connected data. Begin with graph theory, random walks, and PageRank; move through DeepWalk, node2vec, graph convolution, attention, adversarial learning, and graph transformers. Read critically, reproduce results, and develop an original, carefully evaluated research project.

COURSE PREREQUISITE(S)

Required background	Data structures and algorithms; basic discrete mathematics and graph terminology. Linear algebra, probability, calculus, and an introductory machine learning course or equivalent experience. Comfortable Python programming; basic PyTorch experience is strongly recommended. Formal course equivalencies are to be confirmed.

COURSE OBJECTIVES

Course aims	Connect graph structure, random walks, spectral ideas, and representation learning. Develop the ability to read, explain, implement, and critically evaluate graph machine learning research. Carry a focused research question from reproducible baselines to experiments and a written research paper.

Learning Outcomes	
Students will be able to	1. Analyze graph representations, connectivity, centrality, random walks, and Laplacians, and relate them to learning tasks. 2. Derive and implement representative random-walk, convolutional, attention, adversarial, and transformer methods. 3. Critique assumptions, expressivity, scalability, robustness, and evaluation protocols in research papers. 4. Design reproducible node, link, or graph prediction experiments with valid splits, baselines, ablations, and uncertainty reporting. 5. Communicate at least five paper reproductions in a group portfolio and defend individual contributions. 6. Formulate a research question, execute a phased project, and individually write and submit a research paper on the project.

Grading Breakup and Policy
Proposed assessment totals 100%. Five-paper portfolio: 25%. Five paper studies per group, 5% each. For each: understanding and critique 20%, implementation and reproducibility 40%, evaluation 20%, presentation and individual defense 20%. Surprise quizzes: 10%. Short individual quizzes on foundations, assigned readings, and implementation reasoning. Timing is unannounced; scope follows material already assigned. Research participation: 5%. Prepared discussion, constructive peer reviews, and evidence-based questions. Attendance alone earns no separate marks. Midterm examination: 15%. Individual examination on weeks 1-7; proposed week 8, 90 minutes. Concepts, derivations, graph problems, and interpretation of experiments. Phased research project: 25%. Proposal 3%, reproducible baseline 5%, method and pilot 5%, full evaluation and demo 5%, individual research paper 7%. Final examination: 20%. Individual cumulative examination with emphasis on post-midterm material; proposed final examination period, 120 minutes. No separate graded homework or attendance component is proposed. Practical exercises support the paper portfolio and project. Any reuse between the two must be declared and must add a distinct contribution to receive credit. All assessment weights, examination timings, contact hours, and checkpoint weeks are proposed. The instructor will confirm them before teaching begins. Examination aids, exact dates, late-work rules, and quiz make-up arrangements will be announced under university policy.

Examination Detail	
Midterm Exam	Yes: individual examination. Separate course paper; no combined examination proposed. Duration: 90 minutes (proposed). Preferred timing: week 8; exact date to be announced. Coverage: weeks 1-7. Graph reasoning, derivations, model comparisons and experiment interpretation. Permitted aids to be announced.
Final Exam	Yes: individual cumulative examination. Separate course paper. Duration: 120 minutes (proposed). Timing: official final examination period. Coverage: cumulative, with emphasis on advanced methods and research reasoning. Permitted aids to be announced.

Campus supports & Key university policies

Campus Supports

Students are strongly encouraged to meet course instructors and TA's during office hours for assistance in course-content, understand the course's expectations from enrolled students, etc. Beyond the course, students are also encouraged to use a variety of other resources. (Instructors are also encouraged to refer students to these resources when needed.) These resources include Counseling and Psychological Services/CAPS (for mental health), LUMS Medical Center/LMC (for physical health), Office of Accessibility & Inclusion/ OAI (for long-term disabilities), advising staff dedicated to supporting and guiding students in each school, online resources (https://advising.lums.edu.pk/advising-resources), etc. To view all support services, their specific role as well as contact information click here (https://advising.lums.edu.pk/#supportservices).

Academic Honesty/Plagiarism

LUMS has zero tolerance for academic dishonesty. Students are responsible for upholding academic integrity. If unsure, refer to the student handbook and consult with instructors/teaching assistants. To check for plagiarism before essay submission, use similarity@lums.edu.pk. Consult the following resources: 1) Academic and Intellectual Integrity (http://surl.li/gpvwb), and 2) Understanding and Avoiding Plagiarism (http://surl.li/gpvwo).

LUMS Academic Accommodations/ Petitions policy

Long-term medical conditions are accommodated through the Office of Accessibility & Inclusion (OAI). Short-term emergencies that impact studies are either handled by the course instructor or Student Support Services (SSS). For more information, please see Missed Instrument or 'Petition' FAQs for students and faculty (https://rb.gy/8sj1h)

LUMS Sexual Harassment Policy

LUMS and this class are a harassment-free zone. No behavior that makes someone uncomfortable or negatively impacts the class or individual's potential will be tolerated.

To report sexual harassment experienced or observed in class, please contact me. For further support or to file a complaint, contact OAI at oai@lums.edu.pk or harassment@lums.edu.pk. You may choose to file an informal or formal complaint to put an end to the offending behavior. You can also call their Anti-Harassment helpline at 042-35608877 for advice or concerns. *For more information: Harassment, Bullying & Other Interpersonal Misconduct: Presentation (http://surl.li/gpvwt)*

COURSE OVERVIEW			
Week/ Lecture/ Module	Topics	Recommended Readings	Objectives/ Application
1	Graphs as data Graph types, adjacency and incidence matrices, degree, paths, connectivity, graph tasks; Python and sparse graph tools.	Graph theory primer; no paper prerequisite	Build and inspect a small graph; establish the reproduction workflow.
2	Spectral structure and graph descriptors Laplacians, centrality, random walks and PageRank; spectral embeddings and multiscale structural summaries.	P01, P02, P06	Implement PageRank; test damping and disconnected graphs.
3	Walk embeddings and statistical representations DeepWalk, node2vec, effective resistance, distributed and streaming descriptors; learned versus engineered features.	P03, P04, P05, P07, P08	Proposal due; choose and register the five-paper portfolio.
4	From graph signals to GCNs The original graph neural network model, spectral filters, Chebyshev approximations and graph convolution.	P12, P13, P16	Portfolio 1; implement a GCN and audit data leakage.
5	Message passing and attention GraphSAGE, MPNNs and GAT; local spectral augmentation, inductive learning and neighborhood aggregation.	P14, P15, P17, P20	Compare aggregation and attention under matched budgets.
6	Graph prediction and feature construction GIN, pooling, attributed-graph representations and control-informed features; graph-level prediction and fair feature comparisons.	P10, P18, P19, P21, P22	Portfolio 2 and project baseline due.
7	Expressivity, failure modes and evaluation Graph distinguishability, heterophily, oversquashing, benchmark splits and experimental controls.	P24, P25, P26, P48	Design a failure-case experiment; midterm review.
8	Midterm and research clinic Midterm on weeks 1-7; review baselines and experimental plans.	Review and synthesis	Individual examination; no new portfolio deadline.
9	Scaling and self-supervision Streaming and multiscale descriptors, sampled training, graph contrastive learning and control-informed learning backbones.	P09, P11, P27, P28, P29, P30, P31	Portfolio 3; descriptor and augmentation spotlights; profile compute and sparsity.
10	Graph learning in practice GNNs for combinatorial optimization, circuits, code, communication networks and resilient control; relational and temporal modeling.	P32, P33, P34, P35, P36, P37, P38, P39	Method and pilot due; application spotlights and feasibility or leakage checks.

COURSE OVERVIEW			
Week/ Lecture/ Module	Topics	Recommended Readings	Objectives/ Application
11	Graph transformers and structural encodings Graphormer, local/global hybrid architectures and control-based rank encoding; compare structural features and attention at matched compute.	P23, P40, P41	Portfolio 4; compare transformer and GNN at matched compute.
12	Adversarial and generative graph learning GAN principles, GraphGAN, MolGAN and autoregressive graph generation; validity, diversity and mode collapse.	P42, P43, P44, P45	Measure validity, diversity, and mode collapse; distinguish GAN from GAT.
13	Trustworthy graph research Explanations, adversarial robustness, graph unlearning and brain-connectomics evaluation; limitations, privacy and reproducibility.	P46, P47, P49, P50	Portfolio 5; project evaluation and demo due.
14	Research symposium Group project presentations, cross-paper synthesis, reproducibility review.	Project-specific revisits	Peer review drafts and artifacts; address remaining paper coverage.
15	Writing and defense Research argument, limitations, error analysis, individual contribution defense.	All-course synthesis	Individual research paper due; final review.
16	Final examination Cumulative assessment with emphasis on advanced methods and research reasoning.	Review	Final exam, subject to the official examination calendar.

Textbook(s)/Supplementary Readings

The primary reading set is CS4308_Papers.pdf (50 papers, P01-P50), with editable LaTeX in CS4308_Papers.tex. The weekly table uses these stable paper identifiers.

The 50-paper set includes 22 publications coauthored by Mudassir Shabbir on graph learning and its representation foundations, plus 28 influential papers selected for their significance to the field.

William L. Hamilton. Graph Representation Learning. https://www.cs.mcgill.ca/~wlh/grl_book/ Primary optional textbook; graph embeddings and graph neural networks.

Albert-Laszlo Barabasi. Network Science. https://networksciencebook.com/ Optional background for graph structure, centrality and network models.

NetworkX developers. NetworkX documentation. https://networkx.org/documentation/stable/ Graph theory and algorithm exercises.

PyG contributors. PyTorch Geometric documentation. https://pytorch-geometric.readthedocs.io/ Practical graph learning implementations.

OGB team. Open Graph Benchmark. https://ogb.stanford.edu/ Datasets, official splits and evaluators.

Research and practice requirements

Five paper portfolio

Class coverage spans 50 papers: 22 coauthored by Mudassir Shabbir and 28 influential field papers. Lectures, paper spotlights, presentations and practical discussions cover the collection; students implement their five selected papers.

Groups will normally have two students. Each group must understand, implement, evaluate, and present at least five distinct papers from the list. Register choices early so the class covers the full list; the instructor fills coverage gaps and approves overlap.

Choose at least one random-walk or spectral paper, one message-passing or attention paper, one advanced paper, one evaluation or reliability paper, and one paper aligned with the project. For a theoretical or benchmark paper, implement its central diagnostic, counterexample, or evaluation protocol.

Each study includes a concise critical brief, runnable code with a documented environment, a reproduced central result, one controlled ablation or stress test, an experiment log, and a presentation with questions. Small-scale reproductions are acceptable when the scope and departures are declared.

Both partners must understand every selected paper and contribute to implementation and presentation. Use contribution statements and brief individual oral checks. Cite reused code and distinguish the group's implementation from upstream code.

Proposed portfolio checkpoints are weeks 4, 6, 9, 11, and 13. Each checkpoint includes a short live presentation in class or a parallel studio. Recorded full explanations and rotating live discussions keep the workload feasible as enrollment grows.

Phased research project

Week 3 - Question and proposal (3% of course). A two-page proposal: research question, hypothesis, related papers, dataset access and license, metrics, compute budget, and risks.

Week 6 - Reproducible baseline (5% of course). Data audit, frozen splits, simple and competitive baselines, runnable environment, and initial results.

Week 10 - Method and pilot (5% of course). A justified extension or diagnostic, pilot experiments, implementation details, and a revised experiment plan.

Week 13 - Evaluation and demo (5% of course). Ablations, multiple seeds where feasible, uncertainty, runtime and memory measurements, error analysis, and a working demonstration.

Week 15 - Individual research paper (7% of course). Each student submits an individually authored 6-8 page research paper, excluding references and appendices, plus an artifact link and contribution statement. Shared project results must be attributed; individual analysis and writing are required.

A rigorous replication with a meaningful extension, a careful negative result, or a focused new method can all make a strong project. Publication acceptance is not a grading criterion. The required paper is submitted to the course; external conference or journal submission is optional and requires agreement among the authors.

Paper structure: Abstract; introduction and question; related work; method; experimental setup; results and ablations; limitations and ethical considerations; conclusion; references; reproducibility and individual contributions.

Across the project milestones, assess clarity of question, technical correctness, strength of baselines and evaluation, reproducibility, and quality of argument. The individual paper is assessed for synthesis and writing as well as the student's documented contribution; novelty alone does not determine the grade.

Workload and research practice

Plan for approximately 8-10 hours outside class each week; implementation and project deadlines may require more. The instructor will calibrate scope to available compute.

Use fixed train/validation/test splits; select models on validation data and reserve the test set for final evaluation. Document preprocessing, negative sampling, random seeds, hardware, and runtime. Report all attempted conditions relevant to the research claim.

Cite papers, datasets, code, and any AI assistance. Students remain responsible for correctness and must be able to explain and reproduce submitted work. Do not fabricate results or submit generated prose as an individually authored research argument without disclosure and instructor guidance.

Follow dataset licenses and access conditions. Avoid collecting personal data for class projects; use public benchmark or synthetic data with an appropriate research purpose. Do not upload restricted data or credentials to public repositories.

The attached outline retains the university support and policy text supplied in the original template. Students should consult current official university guidance for support, accessibility, academic integrity, and reporting procedures.

Illustrative research projects

Choose a focused question and a feasible experiment. These 20 ideas are starting points, not required projects. Compute estimates are illustrative; agree on a manageable scope with the instructor. Dataset links and expanded descriptions are also available on the course page.

R01 When do biased walks help?

Which graph structures make node2vec's walk bias useful?

Data: Synthetic stochastic block models and Cora. Baselines: DeepWalk, node2vec, degree features plus logistic regression.

Evaluation: Node macro-F1 and accuracy; sweep homophily, walk length, p and q under fixed embedding budgets.

Extension: Relate performance to mixing time and community strength. Compute: CPU-friendly.

Dataset source: https://pytorch-geometric.readthedocs.io/en/latest/generated/torch_geometric.datasets.Planetoid.html

R02 PageRank as a learning prior

Can personalized diffusion improve classification when labels are scarce?

Data: Cora and CiteSeer. Baselines: MLP, GCN, personalized PageRank propagation.

Evaluation: Accuracy versus labeled fraction; fix splits and compare propagation depth and damping.

Extension: Choose damping using local graph structure without test labels. Compute: CPU or small GPU.

Dataset source: https://pytorch-geometric.readthedocs.io/en/latest/generated/torch_geometric.datasets.Planetoid.html

R03 Attention under noisy edges

Does learned graph attention remain useful as irrelevant edges increase?

Data: Cora plus controlled random edge additions. Baselines: GCN, GraphSAGE, GAT.

Evaluation: Accuracy and calibration versus edge corruption; match parameter counts and tuning budgets.

Extension: Compare attention entropy with robustness and test a sparsity regularizer. Compute: Small GPU.

Dataset source: https://pytorch-geometric.readthedocs.io/en/latest/generated/torch_geometric.datasets.Planetoid.html

R04 Learning across unlike neighbors

When do neighborhood aggregation models fail on heterophilous graphs?

Data: Roman-empire and Minesweeper benchmark splits. Baselines: MLP, GCN, GAT and a heterophily-aware model.

Evaluation: Dataset-appropriate accuracy or ROC-AUC; preserve supplied splits and report multiple seeds.

Extension: Adapt the balance of self features and neighbor features. Compute: Small GPU.

Dataset source:
https://pytorch-geometric.readthedocs.io/en/latest/generated/torch_geometric.datasets.HeterophilousGraphDataset.html

R05 How deep can a GNN go?

Which changes preserve predictive information as network depth increases?

Data: Cora and synthetic community graphs. Baselines: Plain GCN, residual GCN and normalization variants.

Evaluation: Accuracy, embedding similarity and Dirichlet energy across depth; control training budget.

Extension: Separate optimization failure from oversmoothing with controlled interventions. Compute: CPU or small GPU.

Dataset source: https://pytorch-geometric.readthedocs.io/en/latest/generated/torch_geometric.datasets.Planetoid.html

Illustrative research projects continued

R06 Rewiring graph bottlenecks

Can a few added edges improve long-range information transfer?

Data: Synthetic trees, barbell graphs and distance-label tasks. Baselines: GCN or MPNN with no rewiring, random rewiring and a topology-aware rule.

Evaluation: Long-range task accuracy, path length and runtime at equal edge budgets.

Extension: Study the trade-off between shortcut benefits and structural distortion. Compute: CPU-friendly.

Dataset source: https://networkx.org/documentation/stable/reference/generators.html

R07 A fair transformer comparison

When does global attention earn its extra compute?

Data: ZINC benchmark graphs. Baselines: GIN, a graph transformer and a local/global hybrid.

Evaluation: Regression MAE, peak memory and runtime; match parameters and training budget.

Extension: Identify graph sizes or structures where global attention helps. Compute: Small GPU; use a declared subset for pilots.

Dataset source: https://pytorch-geometric.readthedocs.io/en/latest/generated/torch_geometric.datasets.ZINC.html

R08 Which positional encoding matters?

How stable are graph transformers to positional encoding choices?

Data: ZINC and synthetic symmetric graphs. Baselines: Transformer without positional encoding, Laplacian encoding and random-walk encoding.

Evaluation: MAE and permutation consistency; vary signs, eigenspaces and graph size.

Extension: Test sign-invariant or noise-robust encoding variants. Compute: Small GPU.

Dataset source: https://pytorch-geometric.readthedocs.io/en/latest/generated/torch_geometric.datasets.ZINC.html

R09 Self-supervision with few labels

Which graph augmentations help when supervision is limited?

Data: Cora and CiteSeer. Baselines: Supervised GCN, DGI and a contrastive graph encoder.

Evaluation: Linear-probe accuracy across label budgets; tune augmentations on validation data only.

Extension: Replace uniform edge dropping with a structure-aware rule. Compute: Small GPU.

Dataset source: https://pytorch-geometric.readthedocs.io/en/latest/generated/torch_geometric.datasets.Planetoid.html

R10 Masked graph reconstruction

Can reconstructing masked features improve robustness to missing attributes?

Data: Cora and PubMed. Baselines: MLP, GCN and GraphMAE-style pretraining.

Evaluation: Classification accuracy and reconstruction loss across random and structured missingness.

Extension: Compare degree-aware masking with uniform masking. Compute: Small GPU.

Dataset source: https://pytorch-geometric.readthedocs.io/en/latest/generated/torch_geometric.datasets.Planetoid.html

Illustrative research projects continued

R11 Predicting future collaborations

How much do graph embeddings improve over simple link heuristics?

Data: ogbl-collab. Baselines: Common neighbors, Adamic-Adar, node2vec and GraphSAGE.

Evaluation: Official Hits@50 with official temporal splits and negative candidates; document allowed edges.

Extension: Add recency-aware edge weights and audit temporal leakage. Compute: Moderate GPU; prototype on a declared subset.

Dataset source: https://ogb.stanford.edu/docs/linkprop/#ogbl-collab

R12 Learning from event streams

When does temporal memory beat a static graph encoder?

Data: Wikipedia temporal interaction data distributed with TGN. Baselines: Static GraphSAGE, a recency heuristic and TGN.

Evaluation: Average precision and ROC-AUC with time-ordered splits and explicit negative sampling.

Extension: Test memory resets, time encodings and unseen-node performance. Compute: Small GPU.

Dataset source: https://github.com/twitter-research/tgn

R13 Relations in a knowledge graph

Do relation-aware messages improve completion beyond embedding models?

Data: FB15k-237. Baselines: DistMult, TransE and R-GCN with a common scoring head where appropriate.

Evaluation: Filtered MRR and Hits@10; prevent inverse-relation leakage and report relation-wise errors.

Extension: Allocate relation parameters by frequency or sparsity. Compute: Small to moderate GPU.

Dataset source: https://pytorch-geometric.readthedocs.io/en/latest/generated/torch_geometric.datasets.RelLinkPredDataset.html

R14 Graph recommendations beyond popularity

Can graph propagation improve recommendation for low-activity users?

Data: MovieLens 100K stable benchmark. Baselines: Popularity, matrix factorization and LightGCN.

Evaluation: Recall@20 and NDCG@20 on a time-ordered split; keep candidate sampling fixed.

Extension: Analyze accuracy and coverage across user-activity groups without inferring sensitive attributes. Compute: CPU or small GPU.

Dataset source: https://grouplens.org/datasets/movielens/100k/

R15 Molecular learning under distribution shift

Do stronger graph encoders generalize to new molecular scaffolds?

Data: ogbg-molhiv. Baselines: Fingerprint classifier, GCN and GIN.

Evaluation: Official ROC-AUC on scaffold splits; add calibration and performance by scaffold frequency.

Extension: Compare pretraining or edge-feature treatments under equal tuning budgets. Compute: Small GPU.

Dataset source: https://ogb.stanford.edu/docs/graphprop/#ogbg-molhiv

Illustrative research projects continued

R16 Generating diverse small graphs

Which generative model best preserves structure without memorization?

Data: Synthetic community and grid graphs. Baselines: Erdos-Renyi or block-model generator, GraphRNN and a small graph GAN.

Evaluation: Validity, uniqueness, novelty, and degree/clustering distribution distances on held-out graphs.

Extension: Diagnose mode collapse as graph size and training diversity change. Compute: Small GPU.

Dataset source: https://networkx.org/documentation/stable/reference/generators.html

R17 Adversarial versus diffusion generation

How do adversarial and diffusion objectives trade quality for cost?

Data: Small synthetic graphs; optional QM9 extension. Baselines: A MolGAN-inspired discrete generator and a small DiGress-style model.

Evaluation: Validity, diversity, novelty, structural fidelity and training/sample time at matched graph sizes.

Extension: If using molecules, add chemical-validity checks and compare conditional generation. Compute: Moderate GPU; synthetic scope recommended.

Dataset source: https://pytorch-geometric.readthedocs.io/en/latest/generated/torch_geometric.datasets.QM9.html

R18 Are graph explanations stable?

Does a model explanation persist across seeds and harmless perturbations?

Data: Synthetic motif graphs with known generating motifs. Baselines: Gradient saliency, random edge masks and GNNExplainer.

Evaluation: Fidelity, sparsity, motif precision/recall and agreement across repeated runs.

Extension: Build a stability-aware explanation selection rule; distinguish explanation from causality. Compute: CPU or small GPU.

Dataset source: https://pytorch-geometric.readthedocs.io/en/latest/generated/torch_geometric.datasets.BAShapes.html

R19 Robustness on a fixed edge budget

Which defenses withstand targeted structural perturbations?

Data: Cora or CiteSeer public benchmark graphs. Baselines: GCN, GAT, edge-drop training and a simple graph denoiser.

Evaluation: Clean and perturbed accuracy across equal budgets; distinguish random from targeted perturbations.

Extension: Test whether defense gains persist on unseen perturbation strategies in an offline benchmark. Compute: Small GPU.

Dataset source: https://pytorch-geometric.readthedocs.io/en/latest/generated/torch_geometric.datasets.Planetoid.html

R20 Do benchmark rankings survive protocol changes?

How sensitive are model rankings to splits, tuning and random seeds?

Data: Cora plus ogbn-arxiv, with a smaller scoped pilot if needed. Baselines: MLP, GCN, GraphSAGE and GAT.

Evaluation: Accuracy distributions, rank stability, runtime and tuning cost; preserve official test protocols.

Extension: Measure how small search budgets alter conclusions and publish a reproducibility checklist. Compute: Small to moderate GPU.

Dataset source: https://ogb.stanford.edu/docs/nodeprop/#ogbn-arxiv